ROOTCAUSE SERVER

19 · Matriz de trazabilidad

Documento: 19-traceability-matrix.md **Versión analizada:** 0.2.0 **Commit:** 767a67b **Fecha del análisis:** 2026-08-26

CONTENIDO

Leyenda de abreviaturas
1 · Detección de superficie expuesta
2 · Detección de intrusión sobre la autenticación
3 · Integridad de archivos críticos
4 · Controles básicos de higiene
5 · Disponibilidad del sensor
6 · Saturación de recursos
7 · Puntuación de postura con honestidad de cobertura
8 · Respuesta guiada, nunca ejecutada
9 · Perímetro propio del plano de control
10 · Autenticación
11 · Seguridad de la consola
12 · Sensor de solo lectura
13 · Protección del transporte del token
14 · Deduplicación y ciclo de vida del incidente
15 · Política de detección versionable
16 · Catálogo de reglas publicado
17 · Evidencia exportable y auditoría
18 · Observabilidad del propio sistema
19 · Compatibilidad de protocolo
20 · Integridad de la cadena de suministro
Cobertura de requisitos formales
Cobertura de ADR
Funcionalidades sin requisito formal asociado

> Permite seguir una funcionalidad desde la interfaz hasta el almacenamiento y sus pruebas. Cada fila enlaza: funcionalidad → requisito o regla de negocio → interfaz o endpoint → módulo → función o tipo → persistencia → prueba → documento → estado de validación.
>
> **Estado de validación:** `Verificado` = se comprobó ejecutando o leyendo el código en este análisis · `Documentado` = existe y está descrito, pero no se ejecutó · `Pendiente` = requiere validación humana.

Leyenda de abreviaturas

Abreviatura	Significado
`core`	`crates/rootcause-core/src/`
`srv`	`crates/rootcause-server/src/`

Abreviatura	Significado
`agt`	`crates/rootcause-agent/src/`
`cp`	`crates/rootcause-server/tests/control_plane.rs`

1 · Detección de superficie expuesta

Aspecto	Valor
Funcionalidad	Detectar servicios alcanzables fuera del host y graduar su riesgo
Requisito	REQ-SEC-001 · «El alcance multiplica el riesgo intrínseco del servicio»
Interfaz	Vista «Superficie expuesta» · `GET /api/v1/exposure` · `GET /api/v1/incidents?category=exposure`
Recolección	`agt/net.rs` → `collect`, `parse_ss`, `parse_netstat_windows`, `parse_netstat_bsd`, `parse_proc_net`
Clasificación	`core/security.rs` → `BindScope::classify`, `port_profile`, `PORT_CATALOG` (48 puertos)
Decisión	`core/policy.rs` → `exposure_severity`, `is_allowed_public`
Detección	`core/detect/exposure.rs` → `detect` · reglas `exposure.service.public`, `exposure.cleartext.protocol`
Respuesta guiada	`core/runbook.rs` → `restrict_listener`
Persistencia	`assets.security_json`, `assets.exposed_services`, `incidents`
Agregación	`srv/api.rs` → `build_exposure`
Pruebas	`loopback_addresses_are_not_exposed`, `wildcard_addresses_are_public`, `unparseable_addresses_fail_towards_the_worse_case`, `public_database_is_critical_and_private_one_is_lower`, `edge_servers_may_publish_http`, `cp::an_exposed_database_becomes_a_critical_incident_with_a_runbook`, `cp::the_exposure_report_lists_the_service_behind_the_port`
Documentos	06 §1, 09, `docs/DETECCION_AMENAZAS.md`
Estado	**Verificado**

2 · Detección de intrusión sobre la autenticación

Aspecto	Valor
Funcionalidad	Detectar ráfagas, barridos, campañas distribuidas y **acceso concedido tras la ráfaga**
Requisito	REQ-SEC-001 · «Correlacionar lo que ningún registro guarda junto»
Interfaz	Vista «Amenazas» · `GET /api/v1/threats` · `GET /api/v1/incidents?category=intrusion`
Recolección	`agt/authlog.rs` → `collect`, `parse_sshd_log`, `parse_windows_security_log`, `aggregate`
Modelo	`core/security.rs` → `AuthEvent`, `AuthOutcome`, `failures_by_source`
Umbrales	`core/policy.rs` → `IntrusionPolicy`
Detección	`core/detect/intrusion.rs` → `authentication` · reglas `.bruteforce`, `.spray`, `.distributed`, `.success_after_burst`
Respuesta guiada	`core/runbook.rs` → `block_source`
Persistencia	`auth_pressure` (PK compuesta), `incidents`

Aspecto	Valor
Agregación	`srv/storage.rs` → `threat_sources`, `total_auth_failures`, `threat_severity`
Pruebas	`failures_are_grouped_by_source_and_successes_ignored`, pruebas de umbral en `intrusion.rs`, `cp::a_brute_force_burst_reaches_the_threat_report`
Documentos	06 §3, 08
Estado	**Verificado**

3 · Integridad de archivos críticos

Aspecto	Valor
Funcionalidad	Detectar cambios y permisos debilitados en archivos que deciden quién entra
Requisito	REQ-SEC-001 · «Huella, nunca contenido»
Interfaz	`GET /api/v1/incidents?category=integrity`
Recolección	`agt/integrity.rs` → `collect`, `fingerprint`, `digest`, `default_paths`
Modelo	`core/security.rs` → `WatchedFile`, `is_world_writable`
Detección	`core/detect/integrity.rs` → `detect` · reglas `integrity.file.changed`, `integrity.file.permissions` · lista `ACCESS_CRITICAL`
Respuesta guiada	`core/runbook.rs` → `verify_file`
Persistencia	`file_baselines` con PK `(agent_id, path)`
Ciclo	`srv/storage.rs` → `file_baseline` (lee) y `update_file_baseline` (escribe **después** de detectar)
Pruebas	`world_writable_mode_is_detected`, pruebas de `integrity.rs`, `cp::a_changed_watched_file_is_detected_only_after_a_baseline_exists`
Documentos	06 §3, 07
Estado	**Verificado**

4 · Controles básicos de higiene

Aspecto	Valor
Funcionalidad	Firewall activo con denegación entrante, parches pendientes, reloj sincronizado
Requisito	REQ-SEC-001
Interfaz	`GET /api/v1/incidents?category=hygiene`
Recolección	`agt/baseline.rs` → `collect`, `firewall`, `pending_updates` y sus siete parsers
Modelo	`core/security.rs` → `FirewallState`
Umbrales	`core/policy.rs` → `HygienePolicy`
Detección	`core/detect/hygiene.rs` → `detect`, `clock` · reglas `.firewall.disabled`, `.updates.pending`, `.clock.skew`
Respuesta guiada	`core/runbook.rs` → `enable_firewall`
Persistencia	`assets.security_json`, `incidents`
Pruebas	Parsers de `baseline.rs` (ufw, firewalld, nft, netsh, socketfilterfw, apt, dnf); umbrales en `hygiene.rs`
Documentos	06 §3, `docs/CROSS_PLATFORM.md`

Aspecto	Valor
Estado	**Verificado**. El conteo de parches solo existe en Linux, declarado como límite

5 · Disponibilidad del sensor

Aspecto	Valor
Funcionalidad	Detectar que un agente dejó de reportar, y cerrarlo cuando vuelve
Requisito	REQ-SEC-001 · «Un agente silenciado no puede reportar que fue silenciado»
Interfaz	`GET /api/v1/incidents?category=availability` · estado del activo en la vista «Activos»
Detección	`core/detect/availability.rs` → `agent_silence`, `silence_fingerprint`
Ejecución	`srv/watchdog.rs` → `run`, `sweep_silence` (cada 30 s)
Persistencia	`assets.last_seen`, `incidents`
Consulta	`srv/storage.rs` → `asset_heartbeats`
Pruebas	`a_reporting_agent_produces_no_finding`, `silence_beyond_the_tolerated_window_is_reported`, `prolonged_silence_escalates`, pruebas de `watchdog.rs`
Documentos	03, 06 §3
Estado	**Verificado**

6 · Saturación de recursos

Aspecto	Valor
Funcionalidad	CPU sostenida, presión de memoria, disco al límite y proyección de horas hasta llenarse
Requisito	REQ-SEC-001 · «Una señal aislada no se presenta como causa confirmada»
Interfaz	Panel · `GET /api/v1/incidents?category=resource` · `GET /api/v1/assets/{id}` (historial)
Recolección	`agt/collector.rs` → `sample` (vía `sysinfo`)
Modelo	`core/models.rs` → `MetricSample`, `validate`
Umbrales	`core/policy.rs` → `ResourcePolicy`
Detección	`core/detect/resource.rs` → `disk`, `runway`, `memory`, `cpu`, `sustained_samples`, `hours_to_full`
Persistencia	`telemetry.sample_json`, `assets.latest_metrics_json`
Historial	`srv/storage.rs` → `recent_samples` (12 muestras, `HISTORY_WINDOW`)
Pruebas	`metric_validation_rejects_impossible_values`, pruebas de racha y proyección en `resource.rs`
Documentos	06 §3
Estado	**Verificado**

7 · Puntuación de postura con honestidad de cobertura

Aspecto	Valor
Funcionalidad	Resumir la defensa en un número **acompañado siempre de lo que no se pudo inspeccionar**
Requisito	**REQ-SEC-003 · Honestidad de la cobertura**
Interfaz	Panel (medidor y lista de brechas) · `GET /api/v1/status` · `GET /api/v1/assets`

Aspecto	Valor
Origen de las brechas	`agt/probe.rs` → `ProbeError::reason` → `core/security.rs` → `CollectionGap`
Cálculo	`core/posture.rs` → `compute`, `compute_now`, `weight`, `dimension_weight`, **techo por peor hallazgo**
Protección del cierre	`srv/api.rs` → `self_healing_categories`
Cierre auditado	`srv/storage.rs` → `auto_resolve`
Persistencia	`assets.posture_score`, `audit_log`
Pruebas	`a_clean_fleet_scores_one_hundred`, `resolved_incidents_do_not_count`, **`cp::a_finding_stays_open_when_its_surface_could_not_be_inspected`**, `cp::a_collection_gap_is_reported_back_to_the_agent`, `cp::a_closed_port_resolves_its_own_incident_on_the_next_cycle`
Documentos	06 §4, 08, REQ-SEC-003
Estado	**Verificado.** El tope de 500 incidentes que truncaba el cálculo (R-01) está corregido: la puntuación se calcula con `COUNT(*)` y `GROUP BY` sobre toda la tabla

8 · Respuesta guiada, nunca ejecutada

Aspecto	Valor
Funcionalidad	Entregar el comando exacto y su forma de revertirlo, sin ejecutarlo jamás
Requisito	REQ-SEC-002 · «Ninguna acción destructiva automática»
Interfaz	Diálogo de incidente en la consola · `GET /api/v1/incidents/{id}/runbook`
Modelo	`core/runbook.rs` → `RunbookStep`, `StepKind`
Generadores	`block_source`, `restrict_listener`, `verify_file`, `enable_firewall`
Persistencia	Dentro de `incidents.incident_json`
Garantía	Prueba `no_destructive_commands` en `runbook.rs`: **hace fallar el build**
Pruebas	`no_destructive_commands`, `cp::an_exposed_database_becomes_a_critical_incident_with_a_runbook`
Documentos	06 §5, `AGENTS.md`
Estado	**Verificado**

9 · Perímetro propio del plano de control

Aspecto	Valor
Funcionalidad	Límite de tasa y bloqueo por dirección, evaluados **antes** de comparar el token
Requisito	REQ-SEC-002 · ADR 0003
Interfaz	Respuestas `429` con `Retry-After` · `control_plane_defense` en `GET /api/v1/threats` · `/metrics`
Mecanismo	`srv/defense.rs` → `Perimeter::check`, `record_failure`, `record_success`, `prune`, `Decision`
Orquestación	`srv/auth.rs` → `require_auth`, `client_address`, `bearer_token`, `tokens_match`
Errores	`srv/error.rs` → `ApiError::throttled`, `ApiError::unauthorized`
Persistencia	`defense_events` (memoria para el estado vivo; base para el registro)
Poda	`srv/watchdog.rs` → `PERIMETER_IDLE` (900 s)

Aspecto	Valor
Pruebas	Pruebas de cubeta y bloqueo en `defense.rs`, `token_comparison_rejects_prefixes_and_accepts_the_exact_secret`, `a_zero_retry_hint_is_never_sent`, `cp::repeated_wrong_tokens_lock_the_caller_out`
Documentos	11, 06 §6-7
Estado	**Verificado**

10 · Autenticación

Aspecto	Valor
Funcionalidad	Token bearer único, comparado en tiempo constante
Requisito	REQ-SEC-002
Interfaz	Cabecera `Authorization` · diálogo de token de la consola
Generación	`srv/main.rs` → comando `token` (dos UUID v4)
Validación de configuración	`srv/config.rs` → `ServeSettings::validate`
Comprobación	`srv/auth.rs` → `tokens_match` con `subtle::ConstantTimeEq`
Estado en el cliente	`console/app.js` → `sessionStorage`
Declaración	`srv/state.rs` → `hardening()` → `GET /api/v1/status`
Persistencia	**Ninguna.** El token nunca se guarda en la base
Pruebas	`a_short_token_is_refused`, `a_padded_token_is_refused_before_it_causes_a_mismatch`, `no_token_is_only_allowed_on_loopback_and_on_purpose`, `only_bearer_tokens_are_read`, `cp::health_is_public_and_the_api_is_not`, `cp::a_wrong_token_is_rejected_and_the_body_says_why`
Documentos	11, 10
Estado	**Verificado**. Sin rotación: ver R-04

11 · Seguridad de la consola

Aspecto	Valor
Funcionalidad	Que un nombre de equipo reportado por un agente no pueda interpretarse como marcado
Requisito	REQ-SEC-002
Capa 1	`srv/headers.rs` → `CONTENT_SECURITY_POLICY`, `SECURITY_HEADERS`, `apply`
Capa 2	`console/app.js` → `el()`, `svg()` y construcción con la API del DOM
Entrega	`srv/ui.rs` → `static_asset`, `is_reserved`, `rust-embed`
Guardián	`scripts/guard_console.py`, ejecutado en CI
Persistencia	Ninguna
Pruebas	`the_policy_forbids_inline_script_and_remote_origins`, `every_declared_header_is_a_valid_header`, `api_paths_never_fall_through_to_the_console`, `the_console_is_embedded_in_the_binary`, `cp::every_response_carries_the_security_headers`
Documentos	11, 06 §11

Aspecto	Valor
Estado	**Verificado** para la CSP y las cabeceras. **Pendiente** una prueba de comportamiento del renderizado: ver R-09

12 · Sensor de solo lectura

Aspecto	Valor
Funcionalidad	Que el agente no pueda modificar el host donde vive
Requisito	REQ-SEC-002 · `AGENTS.md`, «El agente es de solo lectura»
Mecanismo	`agt/probe.rs` → `ALLOWED_PROGRAMS` (11), `is_allowed`, `run` con `stdin` nulo, `stderr` descartado, `kill_on_drop` y tiempo límite
Usuarios	`agt/net.rs`, `agt/authlog.rs`, `agt/baseline.rs`
Modo de auditoría	`agt/main.rs` → `--dry-run`
Persistencia	Ninguna en el host
Pruebas	Lista blanca con rutas absolutas y `.exe`; rechazo de programas no listados; pruebas de `ProbeError::reason`
Documentos	06 §12, 11
Estado	**Verificado**

13 · Protección del transporte del token

Aspecto	Valor
Funcionalidad	Que el token no viaje en claro por una red
Requisito	REQ-SEC-002
Mecanismo	`agt/main.rs` → `validate_transport`
Regla	`https` siempre; `http` solo hacia loopback; fuera de ahí exige `--allow-insecure-http` explícito
Complemento	`srv/headers.rs` → HSTS solo con `X-Forwarded-Proto: https`
Persistencia	Ninguna
Pruebas	Pruebas de `validate_transport` en `agt/main.rs`, `transport_security_is_only_claimed_for_a_year_over_subdomains`
Documentos	11, 02
Estado	**Verificado**

14 · Deduplicación y ciclo de vida del incidente

Aspecto	Valor
Funcionalidad	Un hallazgo persistente es un incidente que se repite, no cien incidentes
Requisito	REQ-SEC-001 · «Los falsos positivos se reconocen sin borrar historial»
Interfaz	Vista «Incidentes» · `POST /api/v1/incidents/{id}/status`
Huella	`core/detect/mod.rs` → `fingerprint`, `merge_by_fingerprint`
Persistencia	`incidents.fingerprint` con restricción `UNIQUE`

Aspecto	Valor
Ciclo	`srv/storage.rs` → `upsert_incident` (reabre, suma ocurrencias, conserva severidad máxima), `update_incident_status`, `auto_resolve`
Auditoría	`audit_log` con `incident.status_changed` e `incident.auto_resolved`
Pruebas	Pruebas de `storage.rs`, `cp::an_incident_can_be_acknowledged_and_the_change_is_audited`, `cp::a_closed_port_resolves_its_own_incident_on_the_next_cycle`
Documentos	06 §9, 07
Estado	**Verificado**

15 · Política de detección versionable

Aspecto	Valor
Funcionalidad	Que los umbrales sean datos versionables, validados antes de aplicarse
Requisito	REQ-SEC-001 · ADR 0002
Interfaz	`rootcause-server policy [--file]` · `GET /api/v1/policy`
Modelo	`core/policy.rs` → `DetectionPolicy` y sus tres sub-políticas, con `deny_unknown_fields`
Validación	`DetectionPolicy::validate`, `DetectionEngine::new`
Carga	`srv/config.rs` → `detection_policy`
Persistencia	Archivo JSON externo; **no** en la base de datos
Pruebas	`default_policy_is_valid`, `inverted_thresholds_are_rejected`, `unknown_policy_fields_are_rejected`, `policy_round_trips_through_json`, `an_invalid_policy_file_stops_the_server_instead_of_being_ignored`, `cp::the_policy_in_force_can_be_read_back`
Documentos	06 §2, 10
Estado	**Verificado**

16 · Catálogo de reglas publicado

Aspecto	Valor
Funcionalidad	Que «qué detecta hoy» salga del código, no de un documento que envejece
Requisito	REQ-SEC-001 · «Las reglas son versionadas y comprobables»
Interfaz	`rootcause-server rules` · `GET /api/v1/rules` · vista «Reglas»
Fuente	`core/detect/mod.rs` → `RULES` (18), `RuleInfo`, `rule()`
Guardián	`scripts/guard_claims.py`, que compara el conteo con los documentos
Persistencia	Ninguna; es una constante del binario
Pruebas	`rule_ids_are_unique_and_namespaced`, `every_rule_declares_a_question_and_a_technique`, `rule_count_matches_the_catalog`, `cp::the_rule_catalog_is_published_with_its_attack_mapping`
Documentos	05, `docs/DETECCION_AMENAZAS.md`
Estado	**Verificado**. `guard_claims.py` ejecutado el 2026-08-26: en verde

17 · Evidencia exportable y auditoría

Aspecto	Valor
Funcionalidad	Entregar un paquete de evidencia y un rastro de quién cambió qué
Requisito	REQ-SEC-001 · «Cada detección identifica activo, momento, regla y evidencia»
Interfaz	`GET /api/v1/export` (NDJSON) · `GET /api/v1/audit` · vista «Sistema»
Generación	`srv/api.rs` → `render_export`, `export`, `audit`
Registro	`srv/storage.rs` → `audit`, `list_audit`
Persistencia	`audit_log` (**no se poda nunca**), `incidents`
Pruebas	`cp::the_evidence_export_is_newline_delimited_json`, `cp::an_incident_can_be_acknowledged_and_the_change_is_audited`
Documentos	09, 07
Estado	**Verificado.** El paquete incluye ahora todos los incidentes y toda la auditoría (R-01 corregido)

18 · Observabilidad del propio sistema

Aspecto	Valor
Funcionalidad	Métricas Prometheus y sondas de salud
Requisito	*No formalizado como REQ.* Capacidad declarada en `docs/CAPABILITIES.md`
Interfaz	`GET /metrics`, `GET /healthz`, `GET /readyz`
Generación	`srv/api.rs` → `render_metrics`, `health`, `ready`
Origen de los datos	`srv/storage.rs` → `counts`, `list_incidents`; `srv/defense.rs` → `locked_sources`, `tracked_clients`
Persistencia	Lectura de `assets`, `incidents`, `defense_events`
Pruebas	`cp::metrics_are_exposed_in_prometheus_format`, `cp::health_is_public_and_the_api_is_not`
Documentos	09, 13
Estado	**Verificado.** Los contadores por categoría salen de un `GROUP BY` sobre toda la tabla (R-01 corregido)

19 · Compatibilidad de protocolo

Aspecto	Valor
Funcionalidad	Que un agente `0.1` siga funcionando contra un servidor `0.2`
Requisito	`AGENTS.md` · «Compatibilidad de protocolo, o migración documentada **y probada**»
Interfaz	`protocol_version` del sobre · `GET /api/v1/status`
Mecanismo	`core/lib.rs` → `PROTOCOL_VERSION`, `SUPPORTED_PROTOCOL_VERSIONS`, `protocol_is_supported`
Compatibilidad de datos	`TelemetryEnvelope.security` opcional; `Incident.category` con `serde(default)`
Compatibilidad de esquema	`migrations/0002_security_surface.sql`: toda columna nullable o con valor por omisión
Pruebas	`the_current_protocol_is_supported`, `cp::an_unsupported_protocol_version_is_refused`, `cp::an_agent_without_a_security_surface_is_told_so`
Documentos	09, 07

Aspecto	Valor
Estado	**Verificado** para el rechazo y para el sobre sin superficie. **Pendiente** una prueba de actualización de esquema con datos: ver 12, propuesta n.º 4

20 · Integridad de la cadena de suministro

Aspecto	Valor
Funcionalidad	Que nada entre en el árbol de dependencias sin licencia revisada, origen conocido y aviso comprobado
Requisito	`AGENTS.md` · «Toda acción pinneada a SHA completo»
Interfaz	Workflows de CI y publicación
Configuración	`deny.toml`, `.cargo/audit.toml`, `Cargo.lock`
Herramientas	`cargo deny`, `cargo audit`, `gitleaks`, `zizmor`, `actionlint`, `trivy`, `hadolint`, CodeQL
Inventario	`scripts/sbom.py` (CycloneDX, solo biblioteca estándar)
Guardián	`scripts/guard_claims.py` para el pinneo a SHA
Exención documentada	RUSTSEC-2023-0071, **con comprobación de CI que falla si deja de ser cierta**
Pruebas	Trabajo «Dependencias y licencias»; `guard_claims.py` ejecutado el 2026-08-26: en verde
Documentos	11, 13
Estado	**Verificado.** `cargo deny --all-features check` → `advisories ok, bans ok, licenses ok, sources ok`; `cargo audit --deny warnings` → 315 dependencias sin hallazgos; `guard_claims.py` confirma el pinneo a SHA

Cobertura de requisitos formales

Requisito	Título	Filas que lo implementan	Estado
REQ-SEC-001	Detección de comportamiento anómalo y actividad maliciosa	1, 2, 3, 4, 5, 6, 14, 15, 16, 17	Cubierto
REQ-SEC-002	Autoprotección y resiliencia	8, 9, 10, 11, 12, 13	Cubierto
REQ-SEC-003	Honestidad de la cobertura	7 (más las brechas de 1, 3, 4)	Cubierto, con R-01 como excepción a revisar

Cobertura de ADR

ADR	Decisión	Dónde se ve
0001	Rust en las tres plataformas	Todo el código; matriz de CI de 3 sistemas
0002	Detección determinista y política versionable	Filas 15 y 16; `core` sin E/S
0003	Perímetro antes del token	Fila 9

Funcionalidades sin requisito formal asociado

Funcionalidad	Observación
Topología por zonas	Capacidad de presentación; sin REQ propio
Métricas Prometheus	Fila 18; declarada como capacidad, no como requisito
Retención de datos	Implementada (`purge`); su política concreta la decide quien opera
Consola embebida	Cubierta indirectamente por REQ-SEC-002 vía la fila 11

Anterior: 18 · Guía para un nuevo desarrollador · **Índice:** README
